Internal soundness of the PA196 proof search: a direct syntactic route into Peano arithmetic

Research note

13 November 2026

Abstract

The published PA196 development proves in PA that literal nonhalting implies Con(PA), but its reverse soundness argument is external. This note gives a paper-level construction for the missing direction: translate the machine's numeric formulas directly into PA, compile each axiom instance into a PA proof, and retain proof witnesses beside the machine's formula registers. The crucial nonlogical template is finite bit comprehension proved by one full-induction instance. No truth predicate, interpretation through the older M6 decoder, or equality of old and recoded binder labels is used. We spell out the positive-length simulation argument needed to extract a source computation from a literal halting trace. This is a mathematical proof construction, not a new kernel-checked Lean certificate.

1 Exact scope and upstream status

All source references in this note refer to James Chang's PA196 revision `407cdc356f6b5866 35e71ac92be394255f4bfea4`. Its theorem `pa_proves_nonhalting_consistency` proves $\mathrm{PA} \vdash \neg H \to \mathrm{Con(PA)}$, where H is the exact arithmetic blank-tape halting sentence. The theorem `machine_nonhalting` is an external theorem in Lean's metatheory. Neither declaration, by itself, supplies $\mathrm{PA} \vdash H \to \neg\,\mathrm{Con(PA)}$. The difference matters for nonstandard proof codes and nonstandard computation lengths.

We use the ordinary, effective first-order proof predicate for PA. A conservative definitional extension by the elementary arithmetic operations used below is harmless: each auxiliary proof can be translated back into the original language by a primitive recursive proof translation whose correctness is provable in PA. The chosen ordinary PA proof calculus must likewise be calibrated to the upstream PA predicate: standard Hilbert/sequent presentations admit primitive recursive mutual proof translations, provably correct in PA. The consistency formula in the conclusion is the upstream one after this fixed calibration. In particular, 2^n and binary-digit tests can be treated as abbreviations for their standard arithmetic graphs; no exponentiation axiom is added to PA.

2 Translation without recoding variable names

Let $p(a,b) = \binom{a+b+1}{2} + b$ and use exactly

$$\mathsf{I}(a,b) = 4p(a,b), \quad \mathsf{M}(a,b) = 4p(a,b)+1, \quad \mathsf{U}(a,B) = 4p(B,a)+2, \quad \mathsf{E}(a,b) = 4p(a,b)+3.$$

The decoder gives code 0 the formula $\top$. For positive code $n = 4p(u,v) + r$, its constructor is implication, membership, universal quantification, or equality according as $r = 0,1,2,3$; the universal case binds variable v and has body code u.

Define $\tau(n)$ directly as an ordinary arithmetic formula:

$$\begin{aligned}
\tau(0) &= 0 = 0, \\
\tau(4p(u,v)) &= \tau(u) \to \tau(v) \qquad (u+v>0), \\
\tau(\mathsf{M}(u,v)) &= \mathrm{Bit}(x_u, x_v), \\
\tau(\mathsf{U}(v,u)) &= \forall x_v\, \tau(u), \\
\tau(\mathsf{E}(u,v)) &= x_u = x_v.
\end{aligned}$$

Here $\mathrm{Bit}(A,B)$ says that the Bth binary digit of A is 1. The numeral indexing a variable is retained unchanged. In particular, if the source gate binds x_c, the translated gate binds x_c even though the Gödel code of $\tau(c)$ will generally not be c.

Lemma 1 (Syntactic bounds)**.** *The translation is primitive recursive. PA proves, uniformly in n, that every source-variable label occurring in $\tau(n)$ is strictly less than n. Consequently x_n does not occur in $\tau(n)$.*

Proof. For $n>0$, both components of the inverse Cantor pairing of $\lfloor n/4 \rfloor$ are at most $\lfloor n/4 \rfloor < n$. Course-of-values recursion on n therefore defines τ; it can instead be implemented by a table of the values at $0,\dots,n$. The same induction proves the bound on the labels at every atomic leaf and at every quantifier. The argument is an induction on finite *syntax codes*; it is not an induction defining arithmetic truth. A literal expansion of Bit may introduce bound auxiliary variables. Choose these from a fresh block above all source labels, and alpha-rename them when embedding a translated subformula. The assertion concerns source labels; the usual free-variable bound, which is the one needed for capture avoidance, is unaffected by these bound auxiliaries. □

There are two small but essential boundary cases. First, $\mathsf{I}(0,0)=0$, so literal compositionality for implication fails only at this pair. Uniformly in a,b, generate a PA proof of $\tau(\mathsf{I}(a,b)) \leftrightarrow (\tau(a) \to \tau(b))$: outside the exception the formulas coincide; at $(0,0)$ use the fixed tautology $\top \leftrightarrow (\top \to \top)$. Formula congruence transports this equivalence through the fixed shapes of the axiom heads, so a nested zero collapse (for example in head 6 or ρ_8) is covered too. A proof translation for modus ponens branches on this case and returns the fixed PA proof of $0=0$. Second, code $1 = \mathsf{M}(0,0)$ is not syntactically the falsum symbol; its translation is $\chi := \mathrm{Bit}(x_0, x_0)$. PA proves

$$\forall z\, \neg\, \mathrm{Bit}(z,z)$$

from $2^z > z$ and the elementary digit bound. Thus a PA proof of $\tau(1)$ can be converted to a PA refutation. Everywhere the source uses $\mathsf{N}B := \mathsf{I}(B,1)$, source conjunction, or source existential quantification, use the fixed ordinary logical derivations justified by $\forall z \neg\, \mathrm{Bit}(z,z)$. They remain valid if a displayed quantifier binds x_0; one must not treat χ as a syntactically closed atom.

3 A uniform PA proof of the guarded gate

Lemma 2 (Uniform finite bit comprehension)**.** *For each arithmetic formula $R(z,\vec{a})$, PA proves*

$$\forall n\ \exists s < 2^n\ \forall z < n\, \big(\mathrm{Bit}(s,z) \leftrightarrow R(z,\vec{a})\big).$$

Moreover, there is a primitive recursive procedure that, given the syntax code of R and a fresh-variable list, outputs a PA proof of this sentence; PA verifies the correctness of that procedure on all formula codes.

Proof. Use the induction formula

$$Q(n) := \exists s < 2^n \; \forall z < n \, (\mathrm{Bit}(s, z) \leftrightarrow R(z, \vec{a})).$$

For $n = 0$ take $s = 0$. Given a witness $s < 2^n$ for $Q(n)$, apply excluded middle to $R(n, \vec{a})$. In its false case retain s; in its true case use $s + 2^n$. Elementary arithmetic proves that addition of 2^n changes just the nth bit of a number below 2^n, that lower bits are unchanged, and that the new number is below 2^{n+1}. These are fixed PA theorems. Hence the step is a fixed logical proof template with R and its capture-avoiding instances substituted in it. Append exactly the induction axiom for Q and modus ponens.

To see the claimed uniformity, implement the generator by term and formula constructors, freshening, capture-avoiding substitution, proof concatenation, and line-number shifts. Each is primitive recursive. The output contains one induction instance for the input formula, not a decision whether that formula is true. PA checks the output by inspecting the local proof rules and the syntactic induction-axiom test. Induction on the input syntax verifies substitution and freshening. Consequently this is a uniform proof *compiler*, not a uniform satisfaction predicate. □

Write $\varphi = \tau(c)$, and assume the machine's guard $c \neq a, b$. The syntactic-bounds lemma ensures x_c is fresh in φ. Up to the fixed translations of the derived connectives, pointed separation is

$$\exists x_c \, (S \wedge P), \qquad \begin{aligned} P &:= \varphi \to \mathrm{Bit}(x_c, x_b), \\ S &:= \forall x_b \big[(\mathrm{Bit}(x_a, x_b) \to (\varphi \to \mathrm{Bit}(x_c, x_b))) \wedge (\mathrm{Bit}(x_c, x_b) \to \varphi) \big]. \end{aligned}$$

To compile a proof, introduce fresh variables for a copy z of x_b and the selected-set code s, and apply Lemma 2 with

$$n = x_a + x_b + 1, \qquad R(z) := (\mathrm{Bit}(x_a, z) \vee z = x_b) \wedge \varphi[z/x_b].$$

The substitution replaces only *free* occurrences of x_b; all auxiliary binders are fresh. The variables x_a, x_b in the first parenthesis here are the outside parameters. Every set bit of x_a has index below n, and $x_b < n$. The resulting s therefore contains every member of x_a satisfying the property, contains no member failing it, and contains the distinguished point x_b whenever that point satisfies it. Taking $x_c = s$ proves $S \wedge P$.

If $a = b$, the occurrence of x_a inside S is also bound by its $\forall x_b$. Then the first antecedent becomes $\mathrm{Bit}(x_b, x_b)$ and is false. The same selected set still proves the second conjunct and the point clause. A proof generator branches on $a = b$ and uses the fixed irreflexivity proof for this case; it must not silently regard x_a as a fixed parameter inside S.

Under $c \neq a, b$, ordinary logical vacuity supplies proofs of

$$\Delta := x_a = x_b \to \forall x_c (x_a = x_b), \qquad V := \mathrm{Bit}(x_a, x_b) \to \forall x_c \, \mathrm{Bit}(x_a, x_b).$$

Let $Q := \forall x_c (S \to \neg P)$. The pointed-separation proof gives $\neg Q$. Propositional logic then gives

$$F(a, b, c) = \neg(\Delta \to (V \to Q)).$$

This is precisely the translated PA196 gate, modulo the fixed code-1 connective translations. If the guard fails, the machine returns code 0, whose translation has a fixed proof.

4 Compiling proofs and machine commands

Theorem 3 (Uniform syntactic soundness)**.** *There is a primitive recursive proof compiler J such that PA proves*

$$\mathrm{NativeProof}(d, n) \longrightarrow \mathrm{Prf}_{\mathrm{PA}}(J(d), \ulcorner \tau(n) \urcorner).$$

In particular, PA proves that a native proof of code 1 *gives a PA refutation. The same conclusion holds for the exact fuelled PA196 checker by induction on its fuel table.*

Proof. Compile each of the six heads with an ordinary first-order logical proof template. Heads 3 and 4 are equality substitution for the formula Bit; head 5 is the existential identity witness. Head 1 is $(\exists x \forall x A) \to A$, head 2 exchanges universal quantifiers, and head 6 distributes universal quantification over modus ponens. Coincident labels are permitted: ordinary formula substitution and alpha-renaming supply the correct instances. The restricted head ρ_8 is a propositional tautology for all four parameters, so its additional numerical guard is harmless to soundness. Compile the guarded gate by the preceding section. Generalization appends universal generalization to a theorem proof; modus ponens concatenates its two translated premise proofs and adds the MP line, except in the $\mathsf{I}(0, 0) = 0$ case already discussed. The proof-checker recursion has exactly these constructors. Induction on a proof tree, or on the fuel rounds of the specified table checker, verifies each output proof by the elementary syntactic checks. □

There is a second route that avoids binding a separate native-proof predicate to the machine: attach a PA-proof witness directly to each live formula. A store (t, w, d, h) carries proofs of $\tau(t)$ and $\tau(h)$, plus a parallel stack of proofs for the formula stack encoded by w. The zero tail has an infinite implicit tail of code-0 formulas with the fixed truth proof. On an AX command, push the saved proof, append generalization of the proof of h at label d, and append the eight uniformly compiled axiom/gate proofs. On successful MP append the translated MP proof; on failed MP use the truth proof. Shift only moves proof witnesses. If its test $t + h = 1$ succeeds, natural arithmetic gives $t = 1$ or $h = 1$, and the corresponding retained proof yields a PA refutation.

All augmented operations are primitive recursive on codes. PA proves their invariant by induction on the length of a coded command execution. This proves

$$\mathrm{PA} \vdash \mathrm{CommandHalt}(0, 0, 0, 0) \to \exists e \ \mathrm{Prf}_{\mathrm{PA}}(e, \ulcorner \bot \urcorner).$$

The argument does not assume native nonrefutability, standard-model soundness, or the existence of an arithmetic truth predicate.

5 Extracting a command history from a literal halt

A forward simulation alone is insufficient unless its positive progress and nonhalt endpoints are retained. The following elementary lemma supplies the reverse direction uniformly over PA models.

Lemma 4 (Bounded reverse extraction)**.** *Suppose two deterministic, arithmetically coded transition systems have a PA-provable refinement relation R with these properties:*

1. *A fixed finite startup reaches a target configuration related to the initial source configuration. It has no halt before its endpoint.*

2. *A related source configuration is either terminal, or has a positive-length target segment to a configuration related to its source successor. The endpoint is a working configuration whenever the successor source configuration is live, and every intermediate edge is a genuine target edge. A live current state is allowed to have a terminal successor.*

3. *Target halt either has no outgoing edge or is absorbing under the total target step. Coded source traces of every specified finite length exist, target step is functional, and every endpoint related to a live source configuration is nonhalted.*

Then PA proves that a target halt implies a source halt. A target halt at T steps forces a terminal source configuration within at most $T + 1$ source transitions after startup.

Proof. All statements can be written in arithmetic using coded finite vectors. Fix a putative target trace of length T to halt. A halt before the fixed startup endpoint contradicts functionality and the startup trace. Suppose the initial source configuration and the configurations after its first $T + 1$ transitions are all nonterminal. Inductively concatenate their target segments. After i source steps the accumulated length is at least i and the endpoint is a working configuration. This is a PA induction on an arithmetically expressible existence-of-coded-segment invariant. At $i = T + 1$ it gives a target trace longer than T. Determinism forces its configuration at time T to be the halt of the given trace. If halt has no outgoing edge, its next edge is impossible. If halt is absorbing, induction from time T forces the endpoint of the longer trace to be halted, contradicting its live endpoint. The same proof is valid for nonstandard T and coded vectors in any PA model, because the induction is an instance of PA's induction scheme, not an external induction on standard naturals. □

For PA196, the necessary strengthening of the published `physical_step` theorem is already visible in its proof: its left branch is taken when the resolved instruction is literally `halt`; call that pre-execution configuration terminal for Lemma 4. One further source step reaches the source halted sentinel. Its right branch is taken otherwise and produces `Progress true` and the successor `Refines` relation. Retain this branch condition, instead of existentially forgetting it. The exact control labels in this part of the development are the auxiliary halt 215 and the literal halt 196. Project each positive segment through the finite state map; the tape and segment length are unchanged. The active-state invariant excludes the nineteen unused auxiliary controls. This uses the checked row binding and cannot identify the two halt numerals by equality.

For the source-to-command layer one must use the absorbing version of Lemma 4: the coded source step satisfies `InternalCodedSourceStep.step_halted`, so control 0 has a self-loop. It is incorrect to claim that this coded halt has no outgoing source edge. Positive progress is also essential: the existing `source_consume_step` may take zero steps when the residual is 0, so it cannot directly support the counting argument.

Instead use the total global scheduler on triples (N, L, S), where N is the candidate register, L the residual, and S is the HFS encoding of the four-register command store. The empty encoding $S = 0$ means halt; a live command store is a nonempty vector and can never have that encoding. For a live triple define

	$L = 0$	$L > 0$
raw	N	$L - 1$
N'	$N + 1$	N

$$L' = \text{residual}(\text{raw}), \qquad S' = \text{CommandStep}(\text{commandAtom}(\text{raw}), S).$$

Set $F(N, L, S) = (N', L', S')$, and make triples with $S = 0$ absorbing. All functions have the explicit arithmetic graphs of `SourceCommandDecodeInternal.lean` and `CommandStep.lean`; thus F and its finite iterations are arithmetically representable and PA-provably total.

Relate this scheduler state to a coded source configuration by the exact `CommandRelation` of `InternalCodedSourceCommandRelation.lean`:

$$(S = 0 \wedge \text{control}(C) = 0) \quad \vee \quad \begin{array}{l} \text{control}(C) = 26, \quad \text{the source has a valid clean nine-register store,} \\ \qquad\qquad r_0 = N, \quad r_8 = L, \quad [r_1, r_2, r_4, r_5] = S. \end{array}$$

The clean condition is $r_3 = r_6 = r_7 = 0$. For $S \neq 0$ only the second branch is possible. When $L = \mathrm{raw} + 1$, the already proved all-model theorem `InternalCodedSourceRootCommand.root_positive_command` gives a source segment of positive length to a configuration related to $F(N, L, S)$. When $L = 0$, the theorem `root_fresh_command` gives the same conclusion and increments N. Both theorems use `InternalCodedSourceDispatchTotal.dispatch_command`, which handles the three command classes. The results use no `not_refutable` premise. If $S' \neq 0$, the endpoint has control 26 and is live; if $S' = 0$, the nonempty-vector fact rules out the live branch, so its control is 0.

The clean initial triple is $(2, 0, [0, 0, 0, 0])$. Suppose its coded source execution halts in T steps but its initial scheduler state and all states after its first $T+1$ scheduler iterations stay live. Iterating the appropriate positive root theorem gives a source trace of length at least $T + 1$ with control 26 at its endpoint. Determinism and absorption at source control 0 force the same endpoint to have control 0, a contradiction. Hence some scheduler step produces $S' = 0$. A live command state can become 0 only in Shift's $t + h = 1$ branch, as is explicit in `CommandStep.stepExpr` and `shift_value`; MP and AX always return four-entry nonempty vectors. Retaining the finite sequence of scheduler command atoms therefore supplies exactly the command history required by the parallel PA-proof-witness invariant. That invariant yields a PA refutation.

5.1 A shorter proof under the consistency hypothesis

The reverse-extraction construction is useful as a reusable lemma, but the desired implication admits a more direct proof. Work *inside PA* and assume Con(PA). This is the antecedent of the implication being proved, not an external soundness assumption. The parallel proof-witness invariant shows that every scheduler command remains live: a successful Shift halt would supply a PA proof of $\tau(1)$ and hence a PA refutation, contrary to the assumed consistency sentence.

Starting from $(2, 0, [0, 0, 0, 0])$, use the appropriate positive root command theorem at each step. PA induction produces, for every n, a coded source trace of length at least n ending at a clean root configuration of control 26 and carrying the proof witnesses. Because coded source halt is absorbing, such a trace never passes through control 0. For each source step, retain the nonhalt branch of `physical_step`; its proof supplies positive physical progress whenever the instruction is not a halt. An instruction that were halt would make the next source control 0, which has just been excluded. Concatenate these physical segments, prepend startup, and project their active controls to the literal 196-state table. For every n, this gives a genuine coded literal trace of length at least n with a working endpoint.

If a literal halt occurred at time T, choose $n = T + 1$ and compare that trace with the longer one. Literal determinism identifies the configuration at time T as halt, while the longer trace has a following genuine transition; the literal halt has no such transition. This contradiction is obtained by arithmetic induction on coded traces and applies also to nonstandard T. Consequently

$$\mathrm{PA} \vdash \mathrm{Con(PA)} \to \neg H.$$

Classical contraposition gives the desired reverse direction $\mathrm{PA} \vdash H \to \neg\,\mathrm{Con(PA)}$. Together with the published internal implication it yields the paper-level conclusion

$$\mathrm{PA} \vdash \neg H \leftrightarrow \mathrm{Con(PA)}.$$

The concrete proof compiler and the strengthened positive simulation lemmas have *not* been entered and kernel-checked in the upstream Lean development by this note. An implementation should export these declarations explicitly; it should not rename the existing one-way theorem or cite external HF soundness as their proof.

6 What the iPRA work can and cannot contribute

The ConciseIPRA archive studies proof presentations equivalent to intuitionistic primitive recursive arithmetic. Replacing the PA196 calculus by one of them changes the consistency strength; consistency of iPRA cannot be substituted for consistency of full PA. The useful transfer is architectural: explicit syntactic constructors, small local certificate rules, retained witnesses, and elimination of redundant axiom heads by actual proof templates. A size comparison of Python checkers is not a state-count bound for the binary tape machine, and does not by itself validate a new TM construction.

7 Membership-head deletion needs a different argument

The two direct membership heads are

$$\mathsf{E}(a,b) \to (\mathsf{M}(d,a) \to \mathsf{M}(d,b)), \qquad \mathsf{E}(a,b) \to (\mathsf{M}(a,d) \to \mathsf{M}(b,d)).$$

Deleting one would shorten the emitter substantially. An ordinary first-order semantic argument using separation suggests that incoming-neighborhood indistinguishability might force outgoing indistinguishability. That is not yet a derivation in this restricted numeric calculus. The published reverse-rewriting template for a property $\chi(t)$ uses the corresponding *forward* rewriting rule to specialize the separated negative property. Applying that same template to the missing argument position is circular. The companion note `membership_independence.tex` strengthens this observation: explicit counterinterpretations show that neither head is derivable from the remaining exact native rules, even with the full guarded gate. A different PA interpretation of a weaker one-head calculus is a separate question and is not excluded by those counterinterpretations. No deletion of either head is justified by the present construction.